



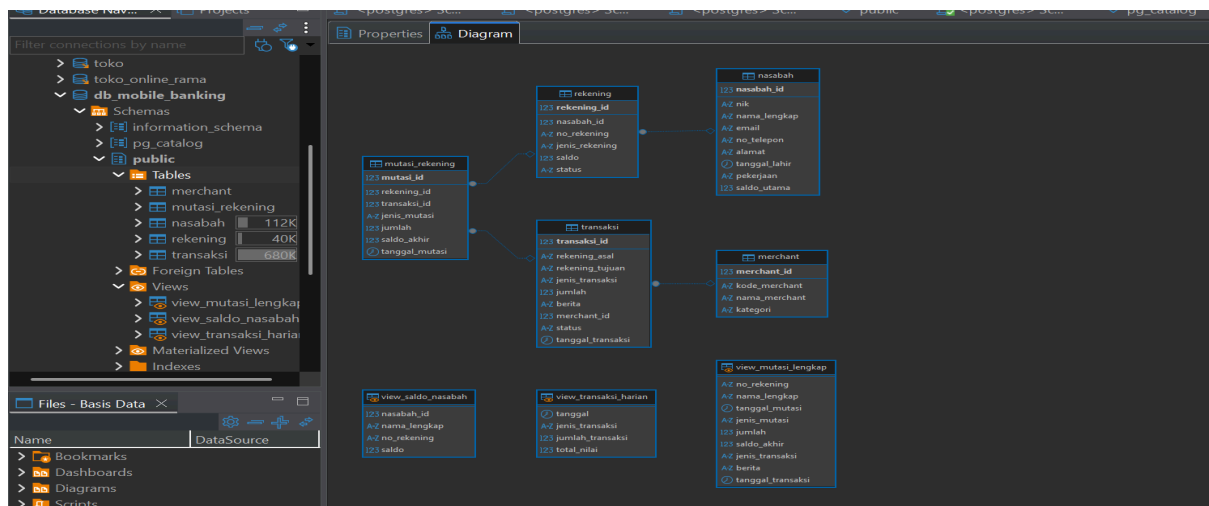
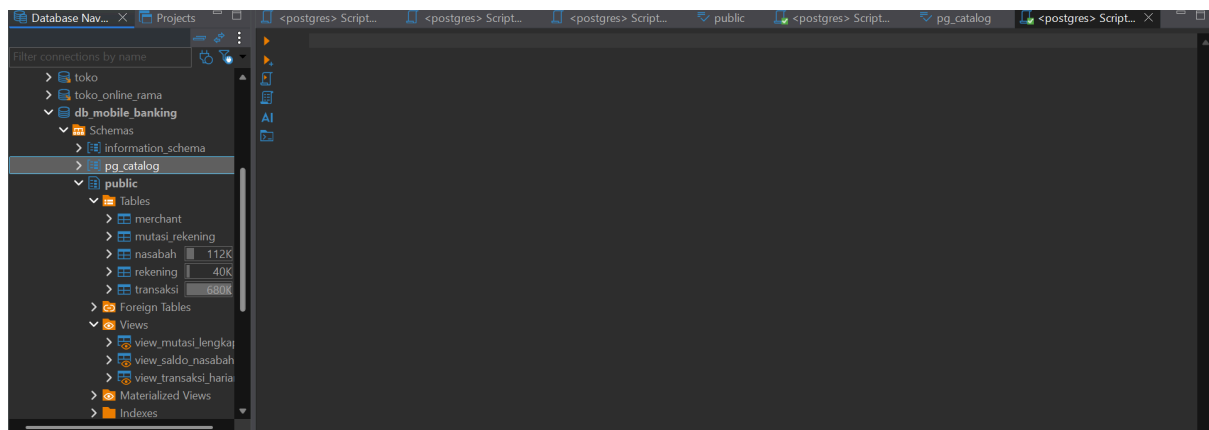
Mata Kuliah : Basis Data Lanjut
Program Studi : D4 – Teknik Informatika
Semester : 3

Kelas : 2E
NIM : 254107020064
Nama : Carlos Fadellilah Rama
Jobsheet Ke- : 7

Laporan Jobsheet

Pertanyaan 1

a. Screenshot hasil diagram yang sudah ada, dan berikan hasil analisis kalian mengenai desain basis data pada db_mobile_banking



b. jelaskan pula relasi yang ada

nasabah ke rekening (One-to-Many / 1: N)

- Relasi: Kolom nasabah_id pada tabel rekening merujuk ke nasabah_id (Primary Key) di tabel nasabah.
- Penjelasan: Satu nasabah dapat memiliki satu atau lebih akun rekening bank (misalnya akun tabungan, giro, dsb.). Namun, satu akun rekening hanya dimiliki oleh satu nasabah.

rekening ke mutasi_rekening (One-to-Many / 1: N)

- Relasi: Kolom rekening_id pada tabel mutasi_rekening merujuk ke rekening_id (Primary Key) di tabel rekening.
- Penjelasan: Satu rekening dapat mencatatkan banyak riwayat mutasi (debit atau kredit). Namun, setiap baris data mutasi hanya milik satu rekening spesifik.

transaksi ke mutasi_rekening (One-to-Many / 1: N)

- Relasi: Kolom transaksi_id pada tabel mutasi_rekening merujuk ke transaksi_id (Primary Key) di tabel transaksi.
- Penjelasan: Satu transaksi (seperti transfer antarrekening) dapat menghasilkan dua catatan mutasi sekaligus (misalnya mutasi debit untuk pengirim dan mutasi kredit untuk penerima).

merchant ke transaksi (One-to-Many / 1: N) [Optional/Nullable]

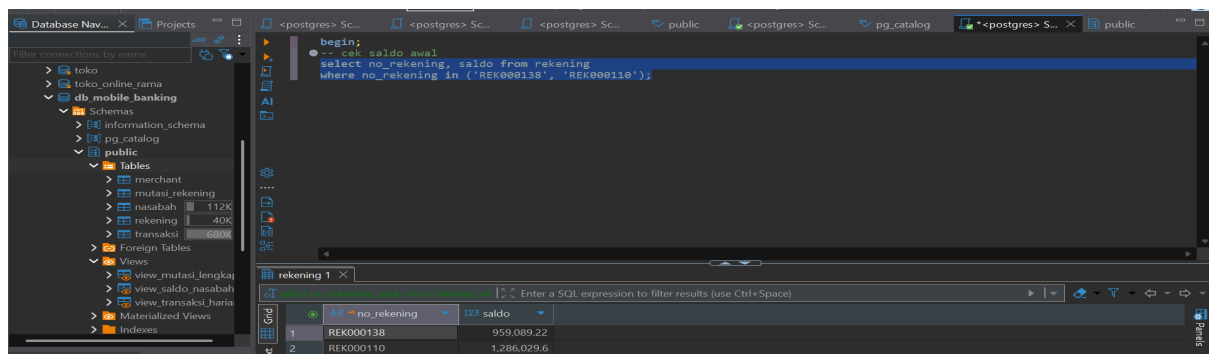
- Relasi: Kolom merchant_id pada tabel transaksi merujuk ke merchant_id (Primary Key) di tabel merchant.
- Penjelasan: Satu *merchant* (toko/penjual) dapat menerima banyak transaksi pembayaran dari pengguna. Nilai ini bersifat opsional (*nullable*) karena transaksi transfer biasa tidak melibatkan merchant.

rekening ke transaksi (Logical Foreign Key / Relasi Konseptual)

- Relasi: Kolom rekening_asal dan rekening_tujuan pada tabel transaksi menyimpan informasi nomor/identitas rekening.
- Penjelasan: Menghubungkan transaksi dengan pihak pengirim dan penerima dana.

Praktikum 1

Pengecekan saldo



Validasi Saldo Manual

The screenshot shows the Database Navigator interface with a PostgreSQL connection. The left sidebar displays the database structure, including schemas (information_schema, pg_catalog, public) and tables (merchant, mutasi_rekening, nasabah, rekening, transaksi). The main editor displays a SQL query for manual balance validation. The query declares a numeric variable `saldo_asal`, selects the current balance from the `rekening` table for rekening number 'REK000138', and then checks if the balance is less than 100,000. If it is, it raises an exception. The query is executed, and the Statistics panel shows 0 updated rows.

```
begin;
-- cek saldo awal
select no_rekening, saldo from rekening
where no_rekening in ('REK000138', 'REK000110');
-- validasi manual
do $$
declare
    saldo_asal numeric;
begin
    select saldo into saldo_asal
    from rekening
    where no_rekening = 'REK000138';

    if saldo_asal < 100000 then
        raise exception 'Saldo tidak cukup. Saldo: %, Butuh: %', saldo_asal, 100000;
    end if;
end $$;
```

Name	Value
Updated Rows	0
Execute time	0.077s
Start time	Wed Oct 07 09:11:07 WIB 2026
Finish time	Wed Oct 07 09:11:07 WIB 2026

Query: -- validasi manual
do \$\$
declare
saldo_asal numeric;
begin

Debit Rekening Asal

The screenshot shows the Database Navigator interface with a PostgreSQL connection. The left sidebar displays the database structure. The main editor displays a SQL query that performs a debit to the original account. The query selects the current balance from the `rekening` table for rekening number 'REK000138', declares a numeric variable `saldo_asal`, and then updates the `rekening` table by subtracting 100,000 from the balance. The query is executed, and the Statistics panel shows 1 updated row.

```
select no_rekening, saldo from rekening
where no_rekening in ('REK000138', 'REK000110');
-- validasi manual
do $$
declare
    saldo_asal numeric;
begin
    select saldo into saldo_asal
    from rekening
    where no_rekening = 'REK000138';

    if saldo_asal < 100000 then
        raise exception 'Saldo tidak cukup. Saldo: %, Butuh: %', saldo_asal, 100000;
    end if;
end $$;
-- debit rekening asal
update rekening
set saldo = saldo - 100000
where no_rekening = 'REK000138';
```

Name	Value
Updated Rows	1
Execute time	0.002s
Start time	Wed Oct 07 09:13:19 WIB 2026
Finish time	Wed Oct 07 09:13:19 WIB 2026

Query: -- debit rekening asal
update rekening
set saldo = saldo - 100000
where no_rekening = 'REK000138'

Kredit Rekening Tujuan

The screenshot shows the Database Navigator interface with a PostgreSQL connection. The left sidebar displays the database structure. The main editor displays a SQL query that performs a credit to the destination account. The query declares a numeric variable `saldo_asal`, selects the current balance from the `rekening` table for rekening number 'REK000138', and then updates the `rekening` table by adding 100,000 to the balance. The query is executed, and the Statistics panel shows 1 updated row.

```
declare
    saldo_asal numeric;
begin
    select saldo into saldo_asal
    from rekening
    where no_rekening = 'REK000138';

    if saldo_asal < 100000 then
        raise exception 'Saldo tidak cukup. Saldo: %, Butuh: %', saldo_asal, 100000;
    end if;
end $$;
-- debit rekening asal
update rekening
set saldo = saldo - 100000
where no_rekening = 'REK000138';
-- kredit rekening tujuan
update rekening
set saldo = saldo + 100000
where no_rekening = 'REK000110';
```

Name	Value
Updated Rows	1
Execute time	0.001s
Start time	Wed Oct 07 09:14:56 WIB 2026
Finish time	Wed Oct 07 09:14:56 WIB 2026

Query: -- kredit rekening tujuan
update rekening
set saldo = saldo + 100000
where no_rekening = 'REK000110'

Pengecekan Saldo di Dalam Transaksi

The screenshot shows a database IDE with a SQL query in the main editor. The query checks the balance of account 'REK000138' and updates it if it's less than 100,000. It also updates account 'REK000110' and checks the balance of both accounts. The results table shows the current balances for both accounts.

```
select saldo into saldo_asal
from rekening
where no_rekening = 'REK000138';

if saldo_asal < 100000 then
    raise exception 'Saldo tidak cukup. Saldo: %, Butuh: %', saldo_asal, 100000;
end if;

end $$;

-- debit rekening asal
update rekening
set saldo = saldo - 100000
where no_rekening = 'REK000138';

-- kredit rekening tujuan
update rekening
set saldo = saldo + 100000
where no_rekening = 'REK000110';

-- cek di dalam transaksi
select no_rekening, saldo from rekening
where no_rekening in ('REK000138', 'REK000110');
```

no_rekening	saldo
REK000138	859,089.22
REK000110	1,386,029.6

Pencatatan Transaksi

The screenshot shows a database IDE with a SQL query in the main editor. The query records a transaction by inserting into the 'transaksi' table. The statistics table shows the execution details of the query.

```
raise exception 'Saldo tidak cukup. Saldo: %, Butuh: %', saldo_asal, 100000;
end if;

end $$;

-- debit rekening asal
update rekening
set saldo = saldo - 100000
where no_rekening = 'REK000138';

-- kredit rekening tujuan
update rekening
set saldo = saldo + 100000
where no_rekening = 'REK000110';

-- cek di dalam transaksi
select no_rekening, saldo from rekening
where no_rekening in ('REK000138', 'REK000110');

-- catat transaksi
insert into transaksi
(rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, tanggal_transaksi, status)
values
('REK000138', 'REK000110', 'TRANSFER', 100000, 'Transfer manual tanpa trigger', '2025-10-07', 'SUCCESS');
```

Name	Value
Updated Rows	1
Execute time	0.04s
Start time	Wed Oct 07 09:17:50 WIB 2026
Finish time	Wed Oct 07 09:17:50 WIB 2026

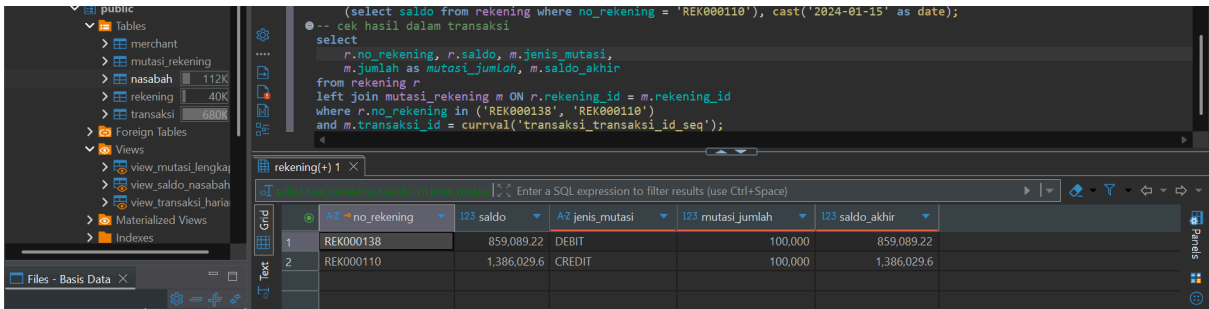
Pencatatan Mutasi Rekening (dengan CTE)

The screenshot shows a database IDE with a SQL query in the main editor. The query records account mutations using Common Table Expressions (CTE) for 'rek_asal' and 'rek_tujuan'. It inserts into the 'mutasi_rekening' table. The statistics table shows the execution details of the query.

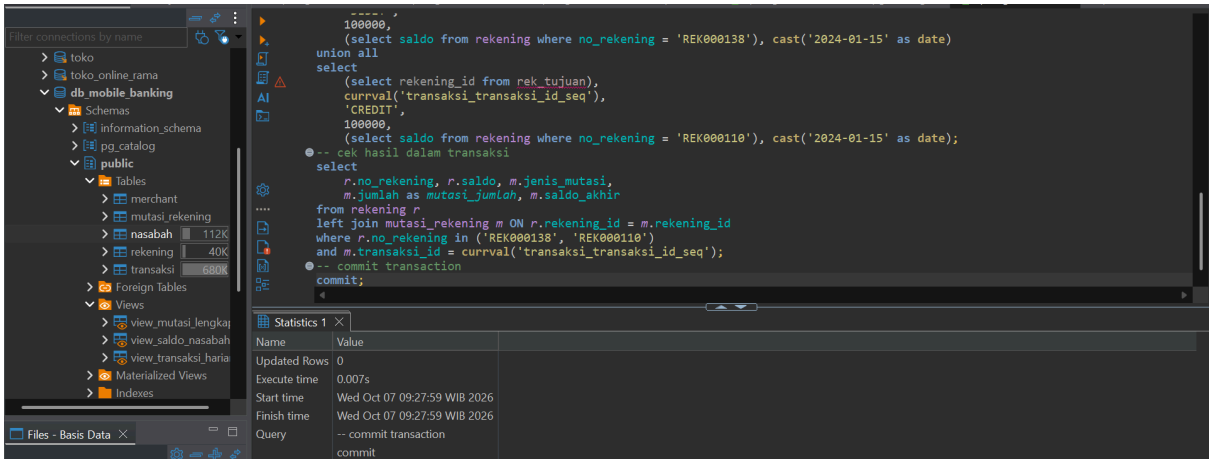
```
-- catat mutasi
with rek_asal as (select rekening_id from rekening where no_rekening = 'REK000138'),
rek_tujuan as (select rekening_id from rekening where no_rekening = 'REK000110')
insert into mutasi_rekening (rekening_id, transaksi_id, jenis_mutasi, jumlah, saldo_akhir, tanggal_mutasi)
select
    (select rekening_id from rek_asal),
    currval('transaksi_transaksi_id_seq'),
    'DEBIT',
    100000,
    (select saldo from rekening where no_rekening = 'REK000138'), cast('2024-01-15' as date)
union all
select
    (select rekening_id from rek_tujuan),
    currval('transaksi_transaksi_id_seq'),
    'CREDIT',
    100000,
    (select saldo from rekening where no_rekening = 'REK000110'), cast('2024-01-15' as date);
```

Name	Value
Updated Rows	2
Execute time	0.049s
Start time	Wed Oct 07 09:24:07 WIB 2026
Finish time	Wed Oct 07 09:24:07 WIB 2026

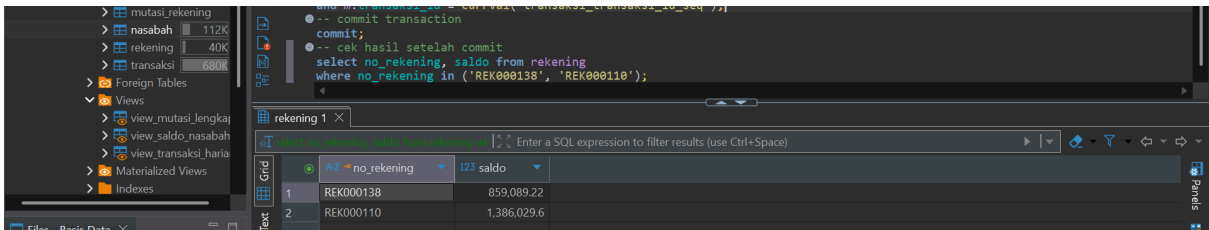
Verifikasi Mutasi dalam Transaksi



Menyimpan Perubahan Permanen (COMMIT)



Verifikasi Akhir Setelah COMMIT



Hasil Pengamatan

Langkah	Operasi	Hasil Sebelum	Hasil Sesudah	Apa yang terjadi?
3	Cek saldo awal	REK000138: [Saldo Awal] REK000110: [Saldo Awal]	REK000138: [Saldo Awal] REK000110: [Saldo Awal]	Menampilkan saldo awal kedua rekening di database sebelum ada perubahan transaksi.
7	Cek dalam transaksi	REK000138: [Saldo Awal] REK000110: [Saldo Awal]	REK000138: [Saldo Awal - 100.000] REK000110:	Saldo berubah di dalam sesi/ koneksi transaksi aktif (debit

			[Saldo Awal + 100.000]	& kredit berhasil secara lokal).
10	Cek mutasi	-	Debit: REK000138 (100.000) Credit: REK000110 (100.000)	Catatan transaksi dan mutasi rekening berhasil dibuat di dalam sesi transaksi (menggunakan CTE & currval).
12	Cek setelah commit	REK000138: [Saldo Awal - 100.000] REK000110: [Saldo Awal + 100.000]	REK000138: [Saldo Awal - 100.000] REK000110: [Saldo Awal + 100.000]	Perubahan saldo dan mutasi tersimpan secara permanen di database setelah perintah COMMIT dieksekusi.

Pertanyaan Analisis

a. Apa yang terjadi jika berhenti di langkah 7 tanpa COMMIT?

- Perubahan saldo (debit dan kredit) hanya bersifat sementara dan hanya terlihat di dalam sesi/koneksi SQL Editor yang sedang aktif.

b. Bagaimana jika terjadi error di langkah 4 atau 5?

- Error di langkah 4 (Validasi Saldo): Blok PL/pgSQL akan melempar *exception* ("Saldo tidak cukup"). Transaksi akan langsung dibatalkan, dan langkah-langkah berikutnya (langkah 5-11) tidak akan diproses.
- Error di langkah 5 (Debit Rekening): Sesi transaksi akan masuk ke status *aborted state* (gagal/batal). Semua query selanjutnya dalam transaksi tersebut akan ditolak oleh PostgreSQL sampai kamu menjalankan perintah ROLLBACK untuk mengembalikan kondisi database ke posisi awal sebelum transaksi dimulai.

c. Sekarang eksekusi perintah ROLLBACK, Apa yang terjadi jika menjalankan ROLLBACK setelah COMMIT?

- Perintah ROLLBACK tidak akan berpengaruh apa pun terhadap data.
- Hal ini terjadi karena perintah COMMIT sudah menyimpan seluruh perubahan secara permanen ke media penyimpanan fisik/database (prinsip *Durability*). Begitu COMMIT selesai dieksekusi, transaksi tersebut dinyatakan resmi berakhir, sehingga tidak ada lagi perubahan gantung yang bisa dibatalkan oleh ROLLBACK. PostgreSQL biasanya hanya akan menampilkan *WARNING/NOTICE* bahwa tidak ada transaksi aktif yang sedang berjalan

Pratikum 2

Memulai Transaksi dan Cek Saldo Awal

The screenshot shows the Database Navigator on the left with the 'db_mobile_banking' database selected. The SQL Editor on the right contains a query to check the initial balances of five accounts. The query results are displayed in a table below the editor.

```
begin;  
select no_rekening, saldo from rekening  
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');
```

no_rekening	saldo
REK000190	8,242,284.23
REK000098	4,048,211.55
REK000149	14,202,793.49
REK000095	2,817,020.19
REK000160	10,284,349.61

Eksekusi Operation 1, Buat SAVEPOINT 1, dan Cek Saldo

The screenshot shows the SQL Editor with a query that performs a transfer, creates a savepoint, and checks the balance. The Statistics panel below the editor shows the execution details.

```
begin;  
select no_rekening, saldo from rekening  
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');  
-- operation 1: transfer a->b (sukses)  
update rekening set saldo = saldo - 150000 where no_rekening = 'REK000098';  
update rekening set saldo = saldo + 150000 where no_rekening = 'REK000149';  
-- buat savepoint  
savepoint after_transfer_1;  
-- cek saldo ditengah transaksi  
select no_rekening, saldo from rekening  
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');
```

Name	Value
Updated Rows	2
Execute time	0.002s
Start time	Wed Oct 07 09:50:04 WIB 2026
Finish time	Wed Oct 07 09:50:04 WIB 2026
Query	-- operation 1: transfer a->b (sukses) update rekening set saldo = saldo - 150000 where no_rekening = 'REK000098'; update rekening set saldo = saldo + 150000 where no_rekening = 'REK000149';

The screenshot shows the SQL Editor with a query that creates a savepoint and checks the balance. The Statistics panel below the editor shows the execution details.

```
begin;  
select no_rekening, saldo from rekening  
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');  
-- operation 1: transfer a->b (sukses)  
update rekening set saldo = saldo - 150000 where no_rekening = 'REK000098';  
update rekening set saldo = saldo + 150000 where no_rekening = 'REK000149';  
-- buat savepoint  
savepoint after_transfer_4;  
-- cek saldo ditengah transaksi  
select no_rekening, saldo from rekening  
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');
```

Name	Value
Updated Rows	0
Execute time	0.001s
Start time	Wed Oct 07 09:50:26 WIB 2026
Finish time	Wed Oct 07 09:50:26 WIB 2026
Query	savepoint after_transfer_1

The screenshot shows the Database Navigator on the left with the 'db_mobile_banking' database selected. The SQL Editor on the right contains a query to check the final balances of five accounts. The query results are displayed in a table below the editor.

```
begin;  
select no_rekening, saldo from rekening  
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');  
-- operation 1: transfer a->b (sukses)  
update rekening set saldo = saldo - 150000 where no_rekening = 'REK000098';  
update rekening set saldo = saldo + 150000 where no_rekening = 'REK000149';  
-- buat savepoint  
savepoint after_transfer_1;  
-- cek saldo ditengah transaksi  
select no_rekening, saldo from rekening  
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');
```

no_rekening	saldo
REK000190	8,242,284.23
REK000095	2,817,020.19
REK000160	10,284,349.61
REK000098	3,898,211.55
REK000149	14,352,793.49

Eksekusi Operation 2 dan Cek Saldo

The screenshot shows a SQL IDE with a database schema on the left and a query editor on the right. The query editor contains the following SQL code:

```
-- operation 1: transfer a->b (sukses)
update rekening set saldo = saldo - 150000 where no_rekening = 'REK000098';
update rekening set saldo = saldo + 150000 where no_rekening = 'REK000149';

-- buat savepoint
savepoint after_transfer_1;

-- cek saldo ditengah transaksi
select no_rekening, saldo from rekening
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');

-- operation 2: transfer c->d
update rekening set saldo = saldo - 2500000 where no_rekening = 'REK000190';
update rekening set saldo = saldo + 2500000 where no_rekening = 'REK000160';

-- cek saldo ditengah transaksi
select no_rekening, saldo from rekening
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');
```

The Statistics panel shows the following results:

Name	Value
Updated Rows	2
Execute time	0.001s
Start time	Wed Oct 07 09:52:14 WIB 2026
Finish time	Wed Oct 07 09:52:14 WIB 2026
Query	-- operation 2: transfer c->d update rekening set saldo = saldo - 2500000 where no_rekening = 'REK000190'; update rekening set saldo = saldo + 2500000 where no_rekening = 'REK000160';

The screenshot shows the same SQL IDE as before, but with the query results displayed in a table view. The table view shows the following data:

no_rekening	saldo
REK000095	2,817,020.19
REK000098	3,898,211.55
REK000149	14,352,793.49
REK000190	5,742,284.23
REK000160	12,784,349.61

Rollback ke SAVEPOINT 1 dan Cek Saldo

The screenshot shows the SQL IDE with a new query in the editor. The query contains the following SQL code:

```
-- cek saldo ditengah transaksi
select no_rekening, saldo from rekening
where no_rekening in('REK000149', 'REK000190', 'REK000160', 'REK000095');

-- operation 2: transfer c->d
update rekening set saldo = saldo - 2500000 where no_rekening = 'REK000190';
update rekening set saldo = saldo + 2500000 where no_rekening = 'REK000160';

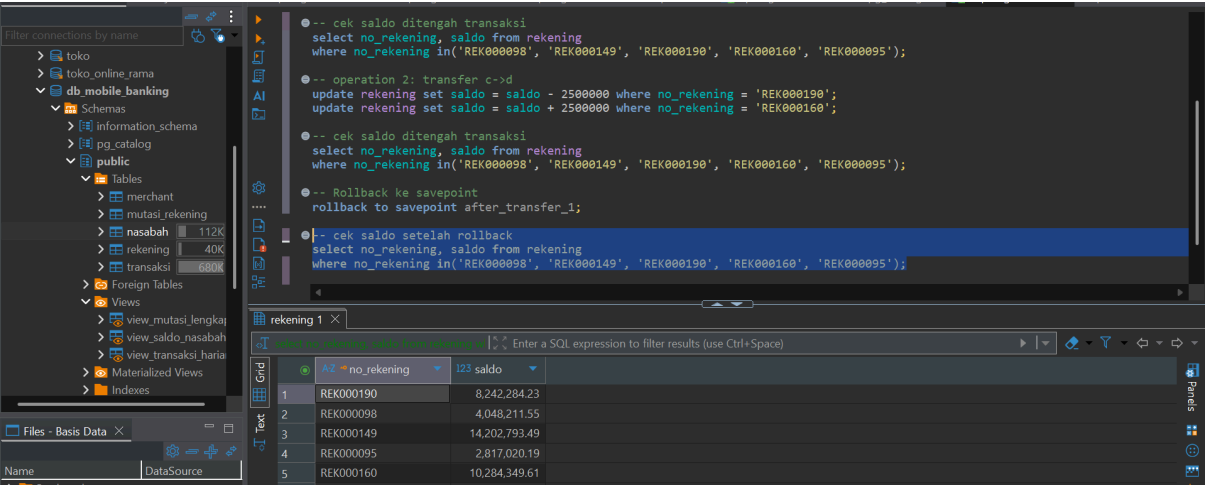
-- cek saldo ditengah transaksi
select no_rekening, saldo from rekening
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');

-- Rollback ke savepoint
rollback to savepoint after_transfer_1;

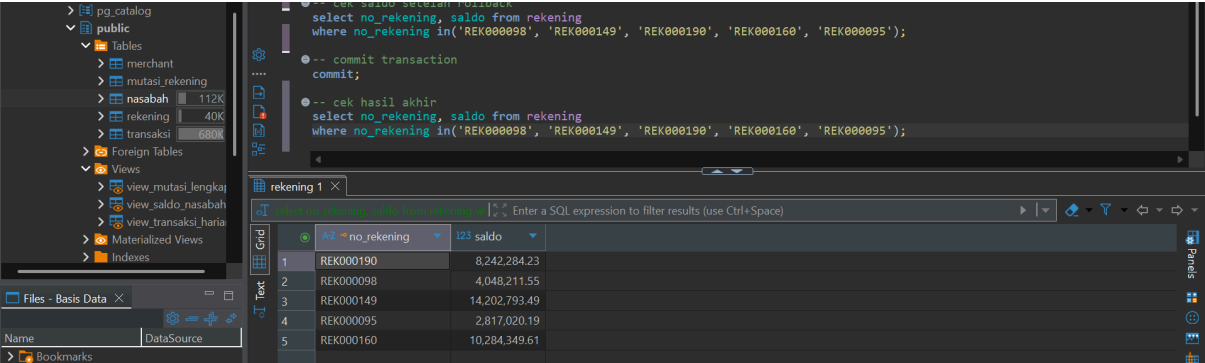
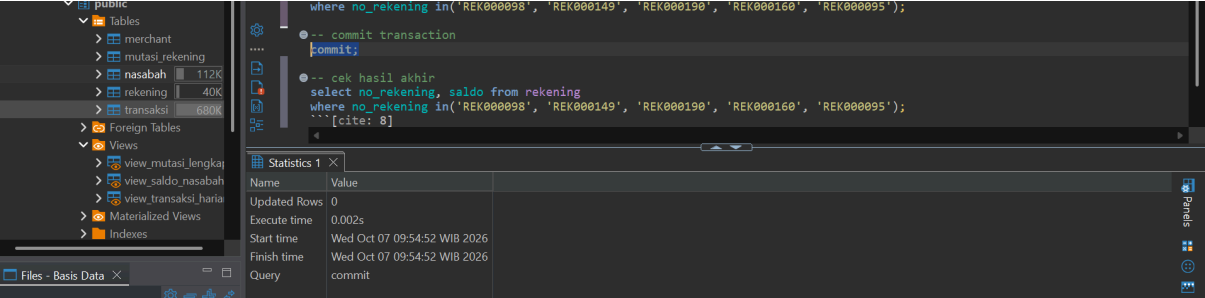
-- cek saldo setelah rollback
select no_rekening, saldo from rekening
where no_rekening in('REK000098', 'REK000149', 'REK000190', 'REK000160', 'REK000095');
```

The Statistics panel shows the following results:

Name	Value
Updated Rows	0
Execute time	0.006s
Start time	Wed Oct 07 09:53:30 WIB 2026
Finish time	Wed Oct 07 09:53:30 WIB 2026
Query	-- Rollback ke savepoint rollback to savepoint after_transfer_1



Commit Transaction dan Cek Saldo Akhir



Hasil Pengamatan

Langkah	Operasi	Hasil Sebelum	Hasil Sesudah	Apa yang terjadi?
3	Cek saldo awal	REK000098: [Saldo Awal] REK000149: [Saldo Awal] REK000190: [Saldo Awal] REK000160: [Saldo Awal]	REK000098: [Saldo Awal] REK000149: [Saldo Awal] REK000190: [Saldo Awal] REK000160: [Saldo Awal]	Menampilkan kondisi saldo awal seluruh rekening target sebelum adanya transaksi.

		[Saldo Awal] REK000095: [Saldo Awal]	Awal] REK000095: [Saldo Awal]	
4	Cek setelah savepoint 1	REK000098: [Saldo Awal] REK000149: [Saldo Awal] REK000160: [Saldo Awal] REK000095: [Saldo Awal]	REK000098: [Saldo Awal - 150.000] REK000149: [Saldo Awal + 150.000] REK000160: [Saldo Awal] REK000095: [Saldo Awal]	Operation 1 (transfer A→B) berhasil dieksekusi secara lokal dan titik after_transfer_1 dibuat.
5	Cek setelah operation 2	REK000098: [Saldo Awal - 150.000] REK000149: [Saldo Awal + 150.000] REK000160: [Saldo Awal] REK000095: [Saldo Awal]	REK000098: [Saldo Awal - 150.000] REK000149: [Saldo Awal + 150.000] REK000190: [Saldo Awal - 2.500.000] REK000160: [Saldo Awal + 2.500.000]	Operation 2 (transfer C→D) berhasil dieksekusi secara lokal di dalam transaksi yang sama.
6	Cek setelah rollback to savepoint 1	REK000098: [Saldo Awal - 150.000] REK000149: [Saldo Awal + 150.000] REK000190: [Saldo Awal - 2.500.000] REK000160: [Saldo Awal + 2.500.000]	REK000098: [Saldo Awal - 150.000] REK000149: [Saldo Awal + 150.000] REK000190: [Saldo Awal] REK000160: [Saldo Awal]	Operation 2 berhasil dibatalkan, sedangkan perubahan dari Operation 1 tetap dipertahankan.
7	Cek setelah commit	REK000098: [Saldo Awal - 150.000] REK000149: [Saldo Awal + 150.000] REK000190: [Saldo Awal] REK000160:	REK000098: [Saldo Awal - 150.000] REK000149: [Saldo Awal + 150.000] REK000190: [Saldo Awal] REK000160: [Saldo Awal]	Hanya hasil Operation 1 yang tersimpan secara permanen ke basis data setelah perintah COMMIT.

		[Saldo Awal]		
--	--	--------------	--	--

a. Bagaimana jika terjadi error di langkah 5?

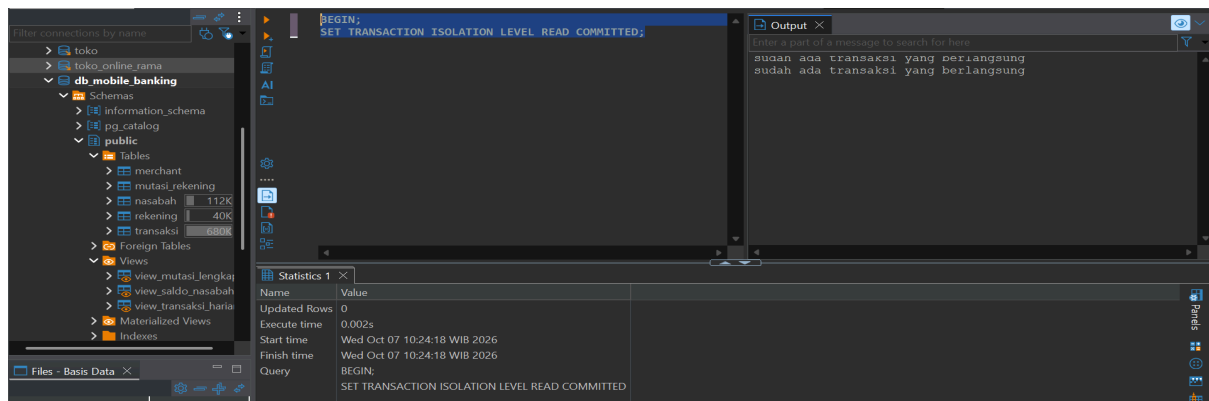
- Jika perintah UPDATE atau operasi lain pada langkah 5 mengalami kesalahan (*error*), transaksi secara keseluruhan tidak akan langsung hancur total.
- Kamu dapat mengeksekusi perintah ROLLBACK TO SAVEPOINT after_transfer_1; untuk membatalkan hanya kesalahan yang terjadi di langkah 5.
- Setelah *rollback* ke *savepoint* tersebut, transaksi akan kembali ke kondisi bersih pada langkah 4, sehingga operasi di langkah 4 (*Operation 1*) tetap aman dan transaksi dapat dilanjutkan atau di-COMMIT.

b. Bagaimana jika kita menggunakan ROLLBACK tanpa menyebut savepoint?

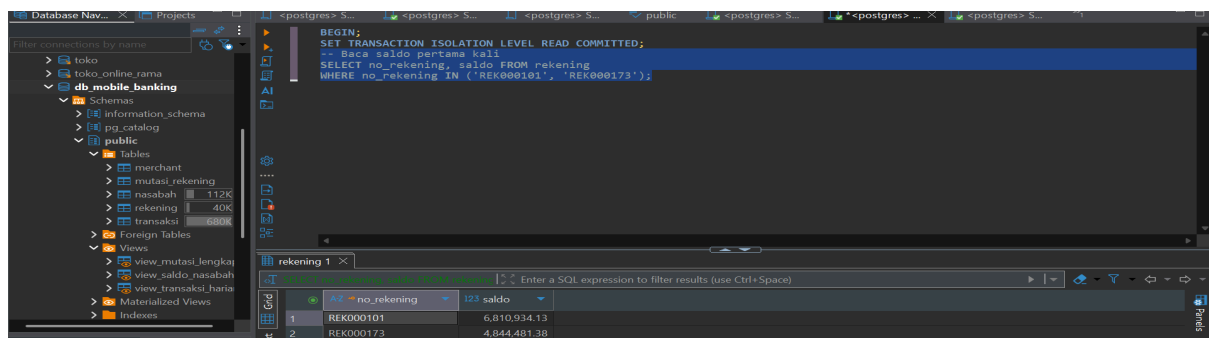
- Jika kamu mengeksekusi perintah ROLLBACK; biasa tanpa menyebutkan nama *savepoint* (misal: ROLLBACK TO SAVEPOINT ...), maka seluruh transaksi dari awal (BEGIN) akan dibatalkan.
- Artinya, tidak hanya *Operation 2* yang dibatalkan, tetapi *Operation 1* yang berada sebelum *savepoint* juga akan ikut terhapus/dibatalkan, sehingga saldo seluruh rekening kembali penuh ke kondisi awal sebelum transaksi dimulai.

Praktikum 3

Window 1: Mulai Transaksi & Set Isolation Level



Window 1: Cek Saldo Awal



Window 2: Mulai Transaksi, Set Isolation Level & Cek Saldo Awal

The screenshot shows the Database Navigator on the left with the 'db_mobile_banking' database selected. The main editor displays the following SQL code:

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SELECT no_rekening, saldo FROM rekening WHERE no_rekening IN ('REK000101', 'REK000173');
```

The Output window on the right shows the message: "sudah ada transaksi yang berlangsung".

Below the SQL editor, a table titled 'rekening 1' is displayed with the following data:

no_rekening	saldo
1 REK000101	6.810.934.13
2 REK000173	4.844.481.38

Window 2: Transfer Uang, Catat Transaksi, dan COMMIT

The screenshot shows the Database Navigator on the left. The main editor displays the following SQL code:

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SELECT no_rekening, saldo FROM rekening WHERE no_rekening IN ('REK000101', 'REK000173');  
-- Lakukan transfer yang mengubah saldo  
UPDATE rekening SET saldo = saldo - 1000000 WHERE no_rekening = 'REK000101';  
UPDATE rekening SET saldo = saldo + 1000000 WHERE no_rekening = 'REK000173';  
-- Catat transaksi  
INSERT INTO transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, tanggal_transaksi)  
VALUES ('REK000101', 'REK000173', 'TRANSFER', 1000000, 'Percobaan isolation level', CURRENT_DATE);  
COMMIT;  
-- Cek saldo setelah commit pada window 2  
SELECT no_rekening, saldo FROM rekening WHERE no_rekening IN ('REK000101', 'REK000173');
```

The Statistics window below the SQL editor shows the following details:

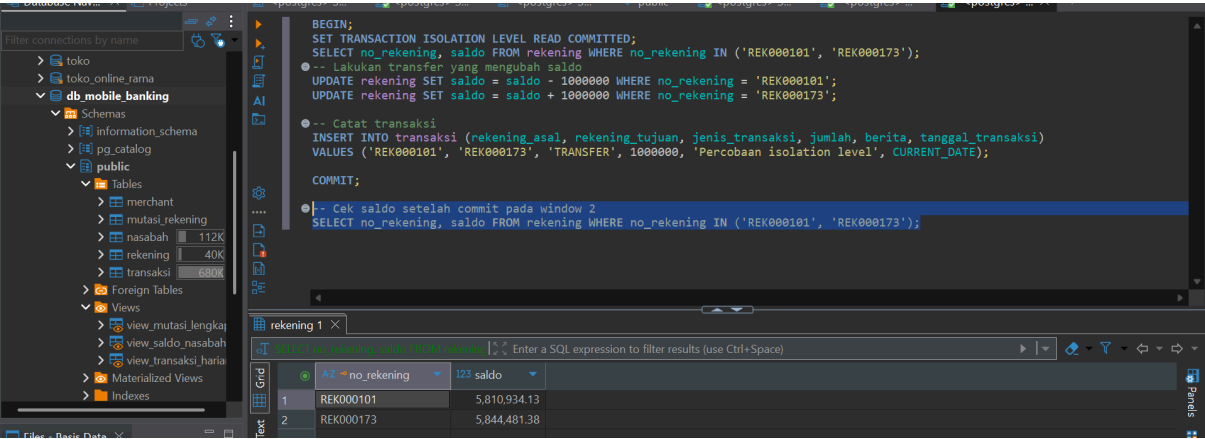
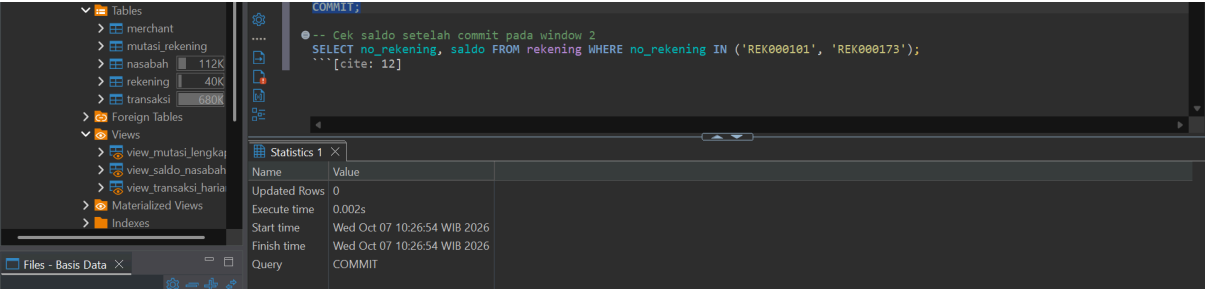
Name	Value
Updated Rows	2
Execute time	0.007s
Start time	Wed Oct 07 10:26:27 WIB 2026
Finish time	Wed Oct 07 10:26:27 WIB 2026
Query	-- Lakukan transfer yang mengubah saldo UPDATE rekening SET saldo = saldo - 1000000 WHERE no_rekening = 'REK000101'; UPDATE rekening SET saldo = saldo + 1000000 WHERE no_rekening = 'REK000173';

The screenshot shows the Database Navigator on the left. The main editor displays the following SQL code:

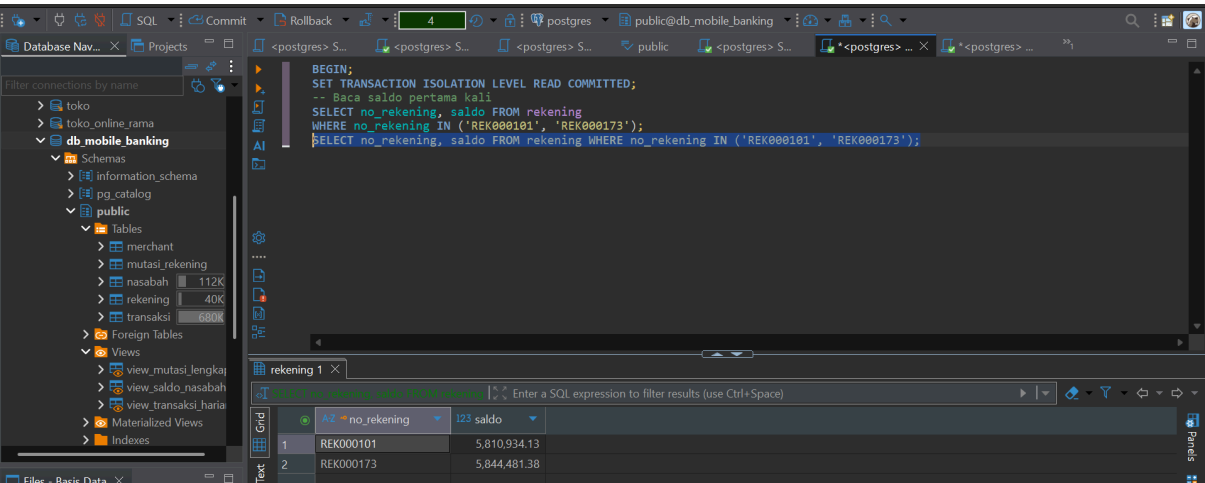
```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SELECT no_rekening, saldo FROM rekening WHERE no_rekening IN ('REK000101', 'REK000173');  
-- Lakukan transfer yang mengubah saldo  
UPDATE rekening SET saldo = saldo - 1000000 WHERE no_rekening = 'REK000101';  
UPDATE rekening SET saldo = saldo + 1000000 WHERE no_rekening = 'REK000173';  
-- Catat transaksi  
INSERT INTO transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, tanggal_transaksi)  
VALUES ('REK000101', 'REK000173', 'TRANSFER', 1000000, 'Percobaan isolation level', CURRENT_DATE);  
COMMIT;  
-- Cek saldo setelah commit pada window 2  
SELECT no_rekening, saldo FROM rekening WHERE no_rekening IN ('REK000101', 'REK000173');
```

The Statistics window below the SQL editor shows the following details:

Name	Value
Updated Rows	1
Execute time	0.008s
Start time	Wed Oct 07 10:26:44 WIB 2026
Finish time	Wed Oct 07 10:26:44 WIB 2026
Query	INSERT INTO transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, tanggal_transaksi) VALUES ('REK000101', 'REK000173', 'TRANSFER', 1000000, 'Percobaan isolation level', CURRENT_DATE)



Window 1: Cek Ulang Saldo



Hasil Pengamatan

Langkah	Operasi	Hasil Sebelum	Hasil Sesudah	Apa yang terjadi?
4	Cek saldo awal window 1	REK000101: [Saldo Awal] REK000173: [Saldo Awal]	REK000101: [Saldo Awal] REK000173: [Saldo Awal]	Window 1 membaca data saldo awal dari basis data di dalam sesi transaksi

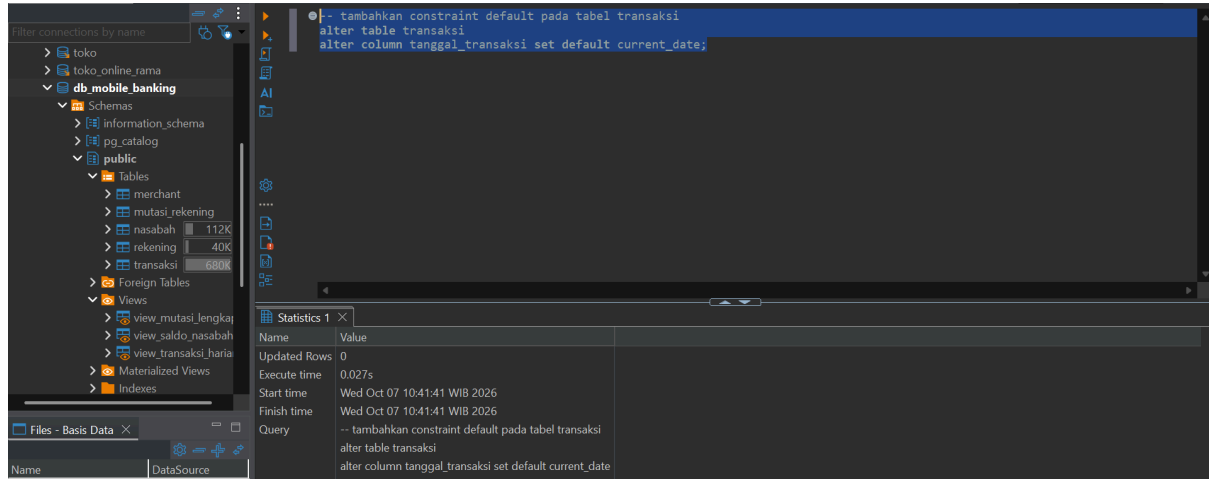
				aktif.
5	Cek saldo awal window 2	REK000101: [Saldo Awal] REK000173: [Saldo Awal]	REK000101: [Saldo Awal] REK000173: [Saldo Awal]	Window 2 membaca data saldo awal yang sama sebelum melakukan perubahan data.
6	Cek saldo setelah commit window 2	REK000101: [Saldo Awal] REK000173: [Saldo Awal]	REK000101: [Saldo Awal - 1.000.000] REK000173: [Saldo Awal + 1.000.000]	Perubahan saldo dan pencatatan transaksi di Window 2 berhasil disimpan permanen ke database.
7	Cek saldo akhir window 1	REK000101: [Saldo Awal] REK000173: [Saldo Awal]	REK000101: [Saldo Awal - 1.000.000] REK000173: [Saldo Awal + 1.000.000]	Window 1 melihat saldo terbaru yang sudah di-COMMIT oleh Window 2 (fenomena Non-Repeatable Read pada level READ COMMITTED).

Pertanyaan Analisis

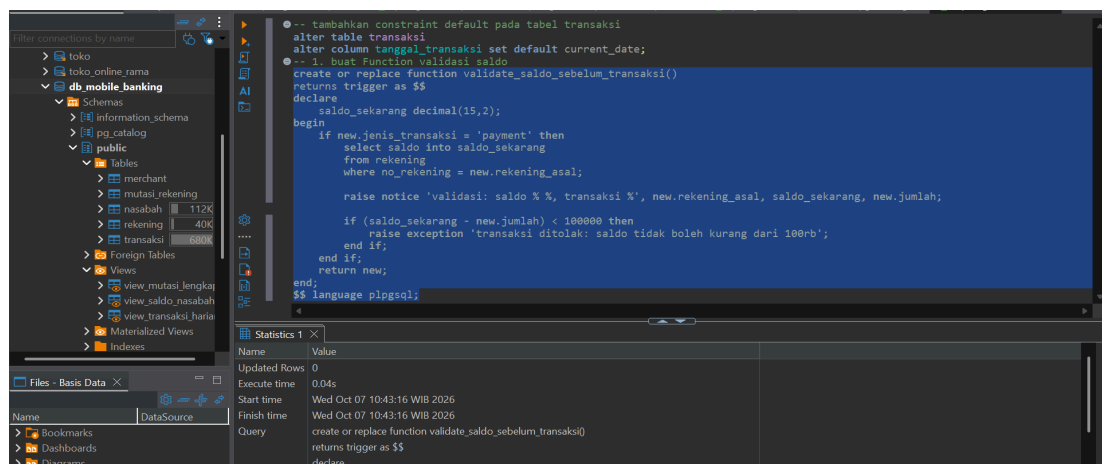
- a. Bagaimana saldo awal dan akhir yang dilihat oleh transaksi pada window 1?
 - Saldo Awal (Langkah 4): Window 1 melihat saldo murni sebelum ada perubahan dari transaksi mana pun.
 - Saldo Akhir (Langkah 7): Window 1 melihat nilai saldo yang sudah berubah (berkurang/bertambah 1.000.000) setelah Window 2 melakukan COMMIT.
- b. Apa yang dapat kamu simpulkan dari isolation level READ COMMITTED?
 - Mencegah Dirty Read: Transaksi tidak dapat membaca data yang sedang diubah oleh transaksi lain jika data tersebut belum di-COMMIT.
 - Membaca Data Terkini: Setelah transaksi lain melakukan COMMIT, perubahan data tersebut akan langsung terlihat oleh transaksi aktif lainnya yang sedang berjalan.
 - Memicu Non-Repeatable Read: Jika query yang sama dijalankan dua kali dalam satu transaksi (seperti pada Window 1), hasil pembacaannya bisa berbeda apabila ada transaksi lain yang melakukan UPDATE dan COMMIT di antara dua pembacaan tersebut.

Praktikum 4

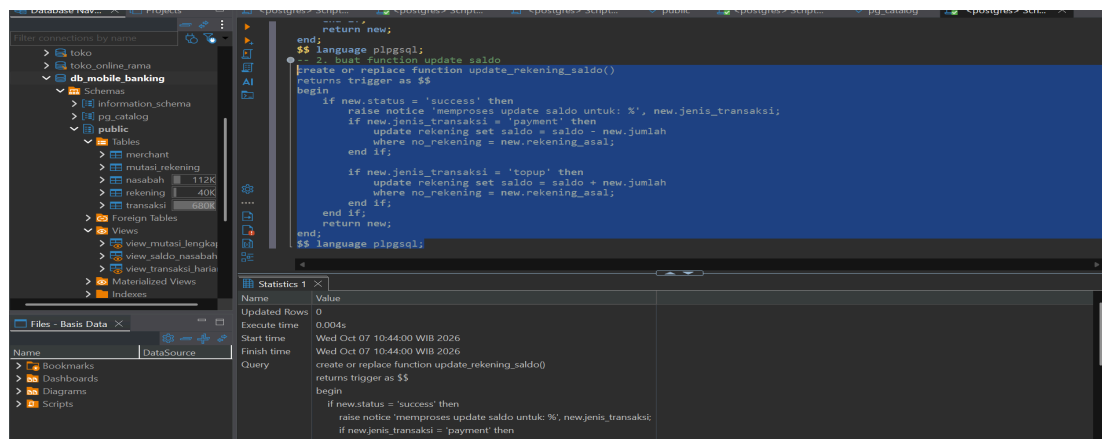
1. Membuka SQL Editor
2. Menambahkan Constraint Default



3. Membuat Function Validasi Saldo Sebelum Transaksi



4. Membuat Function Update Saldo Rekening



5. Membuat Trigger

The screenshot shows the PostgreSQL IDE interface. On the left, the 'db_mobile_banking' database is selected, showing a list of tables including 'merchant', 'mutasi_rekening', 'nasabah', 'rekening', and 'transaksi'. The main editor displays SQL code for creating two triggers. The first trigger, 'trigger_validate_saldo_sebelum_transaksi', is a BEFORE INSERT trigger on the 'transaksi' table that calls the 'validate_saldo_sebelum_transaksi()' function. The second trigger, 'trigger_update_rekening_saldo', is an AFTER INSERT trigger on the 'transaksi' table that calls the 'update_rekening_saldo()' function. The 'Statistics 1' window at the bottom shows the execution details for the first trigger.

```
-- update rekening set saldo = saldo - new.jumlah
-- where no_rekening = new.rekening_asal;
end if;

if new.jenis_transaksi = 'topup' then
    update rekening set saldo = saldo + new.jumlah
    where no_rekening = new.rekening_asal;
end if;
return new;
end;

-- 3. buat trigger
create trigger trigger_validate_saldo_sebelum_transaksi
before insert on transaksi
for each row
execute function validate_saldo_sebelum_transaksi();

create trigger trigger_update_rekening_saldo
after insert on transaksi
for each row
execute function update_rekening_saldo();
```

Name	Value
Updated Rows	0
Execute time	0.008s
Start time	Wed Oct 07 10:45:11 WIB 2026
Finish time	Wed Oct 07 10:45:11 WIB 2026
Query	create trigger trigger_validate_saldo_sebelum_transaksi before insert on transaksi for each row execute function validate_saldo_sebelum_transaksi()

This screenshot shows the same PostgreSQL IDE interface as the previous one, but the SQL code is now being executed. The 'Statistics 1' window at the bottom shows the execution details for the second trigger, 'trigger_update_rekening_saldo'.

```
-- update rekening set saldo = saldo - new.jumlah
-- where no_rekening = new.rekening_asal;
end if;

if new.jenis_transaksi = 'topup' then
    update rekening set saldo = saldo + new.jumlah
    where no_rekening = new.rekening_asal;
end if;
return new;
end;

-- 3. buat trigger
create trigger trigger_validate_saldo_sebelum_transaksi
before insert on transaksi
for each row
execute function validate_saldo_sebelum_transaksi();

create trigger trigger_update_rekening_saldo
after insert on transaksi
for each row
execute function update_rekening_saldo();
```

Name	Value
Updated Rows	0
Execute time	0.005s
Start time	Wed Oct 07 10:45:35 WIB 2026
Finish time	Wed Oct 07 10:45:35 WIB 2026
Query	create trigger trigger_update_rekening_saldo after insert on transaksi for each row execute function update_rekening_saldo()

6. Mengecek Trigger Aktif

The screenshot shows the PostgreSQL IDE interface with a query to check active triggers. The query is: `select trigger_name, event_object table as table_name, action_timing as timing, event_manipulation as event from information_schema.triggers where trigger_schema = 'public';`. The 'triggers 1' window at the bottom shows the results of the query in a table.

```
return new;
end;

-- 3. buat trigger
create trigger trigger_validate_saldo_sebelum_transaksi
before insert on transaksi
for each row
execute function validate_saldo_sebelum_transaksi();

create trigger trigger_update_rekening_saldo
after insert on transaksi
for each row
execute function update_rekening_saldo();

-- cek trigger yang aktif
select
    trigger_name,
    event_object table as table_name,
    action_timing as timing,
    event_manipulation as event
from information_schema.triggers
where trigger_schema = 'public';
```

	A? trigger_name	A? table_name	A? timing	A? event
1	trigger_update_rekening_saldo	transaksi	AFTER	INSERT
2	trigger_validate_saldo_sebelum_transaksi	transaksi	BEFORE	INSERT

7. Persiapan Saldo Awal

The screenshot shows a SQL IDE interface with a database explorer on the left and a query editor on the right. The database explorer shows a schema named 'db_mobile_banking' with tables 'merchant', 'mutasi_rekening', 'nasabah', 'rekening', and 'transaksi'. The 'rekening' table has columns 'no_rekening', 'saldo', and 'jenis_rekening'. The 'transaksi' table has columns 'no_transaksi', 'rekening_asal', 'rekening_tujuan', 'jenis_transaksi', 'jumlah', 'berita', 'merchant_id', and 'status'.

The query editor contains the following SQL code:

```
-- cek trigger yang aktif
select
    trigger_name,
    event_object_table as table_name,
    action_timing as timing,
    event_manipulation as event
from information_schema.triggers
where trigger_schema = 'public';

-- persiapan saldo awal
update rekening set saldo = 1000000
where no_rekening in ('REK000114', 'REK000180');

-- verifikasi saldo awal
select no_rekening, saldo from rekening
where no_rekening in ('REK000114', 'REK000180');
```

The 'Statistics 1' panel shows the execution results:

Name	Value
Updated Rows	2
Execute time	0.012s
Start time	Wed Oct 07 10:47:10 WIB 2026
Finish time	Wed Oct 07 10:47:10 WIB 2026
Query	update rekening set saldo = 1000000 where no_rekening in ('REK000114', 'REK000180')

The 'rekening 1' panel shows the results of the verification query:

no_rekening	saldo
REK000180	1,000,000
REK000114	1,000,000

8. Simulasi Topup Saldo (Transaksi Sukses)

The screenshot shows a SQL IDE interface with a database explorer on the left and a query editor on the right. The database explorer shows a schema named 'db_mobile_banking' with tables 'merchant', 'mutasi_rekening', 'nasabah', 'rekening', and 'transaksi'. The 'rekening' table has columns 'no_rekening', 'saldo', and 'jenis_rekening'. The 'transaksi' table has columns 'no_transaksi', 'rekening_asal', 'rekening_tujuan', 'jenis_transaksi', 'jumlah', 'berita', 'merchant_id', and 'status'.

The query editor contains the following SQL code:

```
-- cek trigger yang aktif
select
    trigger_name,
    event_object_table as table_name,
    action_timing as timing,
    event_manipulation as event
from information_schema.triggers
where trigger_schema = 'public';

-- persiapan saldo awal
update rekening set saldo = 1000000
where no_rekening in ('REK000114', 'REK000180');

-- verifikasi saldo awal
select no_rekening, saldo from rekening
where no_rekening in ('REK000114', 'REK000180');

-- simulasi topup saldo cukup
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000180', 'REK000114', 'topup', 200000, 'test topup', null, 'success');

-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening = 'REK000180';
select * from transaksi where rekening_asal = 'REK000180' order by transaksi_id desc limit 1;
```

The 'Statistics 1' panel shows the execution results:

Name	Value
Updated Rows	1
Execute time	0.01s
Start time	Wed Oct 07 10:50:43 WIB 2026
Finish time	Wed Oct 07 10:50:43 WIB 2026
Query	insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status) values ('REK000180', 'REK000114', 'topup', 200000, 'test topup', null, 'success')

The 'rekening 1' panel shows the results of the verification query:

no_rekening	saldo
REK000180	1,200,000

```

select no_rekening, saldo from rekening
where no_rekening in ('REK000114', 'REK000180');
-- simulasi topup saldo cukup
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000180', 'REK000180', 'topup', 200000, 'test topup', null, 'success');
-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening = 'REK000180';
select * from transaksi where rekening_asal = 'REK000180' order by transaksi_id desc limit 1;

```

transaksi_id	rekening_asal	rekening_tujuan	jenis_transaksi	jumlah	berita	merchant_id
5,004	REK000180	REK000180	topup	200,000	test topup	[NULL]

9. Simulasi Payment Saldo Cukup (Transaksi Sukses)

```

where trigger_schema = 'public';
-- persiapan saldo awal
update rekening set saldo = 1000000
where no_rekening in ('REK000114', 'REK000180');
-- verifikasi saldo awal
select no_rekening, saldo from rekening
where no_rekening in ('REK000114', 'REK000180');
-- simulasi topup saldo cukup
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000180', 'REK000180', 'topup', 200000, 'test to
-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening =
select * from transaksi where rekening_asal = 'REK000180' o
-- payment dengan saldo cukup
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000114', null, 'payment', 150000, 'pembayaran a
-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening =
select * from transaksi where rekening_asal = 'REK000114' o

```

Name	Value
Updated Rows	1
Execute time	0.009s
Start time	Wed Oct 07 10:52:08 WIB 2026
Finish time	Wed Oct 07 10:52:08 WIB 2026
Query	insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status) values ('REK000114', null, 'payment', 150000, 'pembayaran aman', 3, 'success')

```

values ('REK000114', null, 'payment', 150000, 'pembayaran aman', 3, 'success');
-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening = 'REK000114';
select * from transaksi where rekening_asal = 'REK000114' order by tanggal_transaksi desc;

```

AZ no_rekening	saldo
REK000114	850,000

```

where trigger_schema = 'public';
-- persiapan saldo awal
update rekening set saldo = 1000000
where no_rekening in ('REK000114', 'REK000180');
-- verifikasi saldo awal
select no_rekening, saldo from rekening
where no_rekening in ('REK000114', 'REK000180');
-- simulasi topup saldo cukup
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000180', 'REK000180', 'topup', 200000, 'test topup', null, 'success');
-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening = 'REK000180';
select * from transaksi where rekening_asal = 'REK000180' order by transaksi_id desc limit 1;
-- payment dengan saldo cukup
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000114', null, 'payment', 150000, 'pembayaran aman', 3, 'success');
-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening = 'REK000114';
select * from transaksi where rekening_asal = 'REK000114' order by tanggal_transaksi desc;

```

transaksi_id	rekening_asal	rekening_tujuan	jenis_transaksi	jumlah	berita	merchant_id
5,005	REK000114	[NULL]	payment	150,000	pembayaran aman	3
156	REK000114	REK000114	TOPUP	248,259.55	Topup saldo	[NULL]
1,969	REK000114	REK000101	TRANSFER	447,921.38	Transfer ke REK000101	[NULL]
1,467	REK000114	REK000170	TRANSFER	142,484.1	Transfer ke REK000170	[NULL]
4,172	REK000114	[NULL]	WITHDRAWAL	303,071.07	Tarik tunai	[NULL]
2,524	REK000114	[NULL]	PAYMENT	280,587.64	Pembayaran Tokopedia	1

10. Simulasi Payment Saldo Tidak Cukup (Transaksi Gagal)

The screenshot shows a database management tool interface. On the left, a tree view displays the database structure, including tables like 'merchant', 'mutasi_rekening', 'nasabah', 'rekening', and 'transaksi'. The main panel displays a SQL query and its execution results.

```
-- payment dengan saldo cukup
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000114', null, 'payment', 150000, 'pembayaran aman', 3, 'success');

-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening = 'REK000114';
select * from transaksi where rekening_asal = 'REK000114' order by tanggal_transaksi desc;

-- PAYMENT dengan saldo < 100rb
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000114', NULL, 'PAYMENT', 950000, 'Payment berbahaya', 3, 'SUCCESS');
```

The execution results show a single row with the following details:

Name	Value
Updated Rows	1
Execute time	0.002s
Start time	Wed Oct 07 10:53:39 WIB 2026
Finish time	Wed Oct 07 10:53:39 WIB 2026
Query	-- PAYMENT dengan saldo < 100rb insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status) values ('REK000114', NULL, 'PAYMENT', 950000, 'Payment berbahaya', 3, 'SUCCESS')

11. Pengecekan Saldo Akhir dan Catatan Transaksi

The screenshot shows a database management tool interface. On the left, a tree view displays the database structure, including tables like 'pg_catalog', 'public', 'merchant', 'mutasi_rekening', 'nasabah', 'rekening', and 'transaksi'. The main panel displays a SQL query and its execution results.

```
select no_rekening, saldo from rekening where no_rekening = 'REK000114';
select * from transaksi where rekening_asal = 'REK000114' order by tanggal_transaksi desc;

-- PAYMENT dengan saldo < 100rb
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000114', NULL, 'PAYMENT', 950000, 'Payment berbahaya', 3, 'SUCCESS');

-- cek saldo akhir
select no_rekening, saldo from rekening where no_rekening in ('REK000114', 'REK000180');

-- cek transaksi
select rekening_asal, jenis_transaksi, jumlah, status, tanggal_transaksi
from transaksi
where rekening_asal in ('REK000114', 'REK000180')
order by tanggal_transaksi desc;
```

The execution results show a table with the following data:

no_rekening	saldo
REK000114	850.000
REK000180	1.200.000

The screenshot shows a database management tool interface. On the left, a tree view displays the database structure, including tables like 'toko', 'toko_online_rama', 'db_mobile_banking', 'information_schema', 'pg_catalog', 'public', 'merchant', 'mutasi_rekening', 'nasabah', 'rekening', and 'transaksi'. The main panel displays a SQL query and its execution results.

```
-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening = 'REK000180';
select * from transaksi where rekening_asal = 'REK000180' order by transaksi_id desc limit 1;

-- payment dengan saldo cukup
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000114', null, 'payment', 150000, 'pembayaran aman', 3, 'success');

-- cek hasil (jalankan query satu per satu)
select no_rekening, saldo from rekening where no_rekening = 'REK000114';
select * from transaksi where rekening_asal = 'REK000114' order by tanggal_transaksi desc;

-- PAYMENT dengan saldo < 100rb
insert into transaksi (rekening_asal, rekening_tujuan, jenis_transaksi, jumlah, berita, merchant_id, status)
values ('REK000114', NULL, 'PAYMENT', 950000, 'Payment berbahaya', 3, 'SUCCESS');

-- cek saldo akhir
select no_rekening, saldo from rekening where no_rekening in ('REK000114', 'REK000180');

-- cek transaksi
select rekening_asal, jenis_transaksi, jumlah, status, tanggal_transaksi
from transaksi
where rekening_asal in ('REK000114', 'REK000180')
order by tanggal_transaksi desc;
```

The execution results show a table with the following data:

rekening_asal	jenis_transaksi	jumlah	status	tanggal_transaksi
REK000180	topup	200.000	success	2026-10-07
REK000114	PAYMENT	950.000	SUCCESS	2026-10-07
REK000114	payment	150.000	success	2026-10-07
REK000180	WITHDRAWAL	167.125.75	SUCCESS	2025-10-05
REK000114	TOPUP	248.259.55	SUCCESS	2025-10-04
REK000114	TRANSFER	142.484.1	SUCCESS	2025-10-03
REK000114	TRANSFER	447.921.38	SUCCESS	2025-10-03

Hasil Pengamatan

Langkah	Operasi	Hasil Sebelum	Hasil Sesudah	Apa yang terjadi?
6	Cek trigger aktif	-	Menampilkan daftar trigger: - trigger_validate_saldo_sebelum_transaksi (BEFORE INSERT) - trigger_update_rekening_saldo (AFTER INSERT)	Memastikan kedua fungsi trigger telah berhasil terpasang dan aktif pada tabel transaksi.
7	Set saldo awal	REK000114: [Saldo Lama] REK000180: [Saldo Lama]	REK000114: 1.000.000 REK000180: 1.000.000	Saldo kedua rekening berhasil diatur menjadi 1.000.000 sebagai data awal pengujian.
8	Cek transaksi topup berhasil	REK000180: 1.000.000	REK000180: 1.200.000	Transaksi topup sebesar 200.000 berhasil di-INSERT dan trigger AFTER INSERT otomatis menambah saldo REK000180 menjadi 1.200.000.
9	Cek transaksi payment berhasil	REK000114: 1.000.000	REK000114: 850.000	Transaksi payment 150.000 lolos validasi BEFORE INSERT (sisanya saldo $850.000 > 100.000$), lalu trigger AFTER INSERT otomatis memotong saldo REK000114.
10	Payment gagal	REK000114: 850.000	REK000114: 850.000	Transaksi payment sebesar 950.000 memicu exception pada trigger BEFORE INSERT karena sisa saldo

				menjadi di bawah 100.000. Data transaksi ditolak dan dibatalkan.
11	Cek saldo akhir & transaksi akhir	REK000114: 850.000 REK000180: 1.200.000	REK000114: 850.000 REK000180: 1.200.000	Saldo akhir mencerminkan transaksi yang sukses saja. Transaksi yang gagal di langkah 10 tidak tercatat di tabel transaksi maupun memotong saldo.

Pertanyaan Analisis

a. Perbedaan Fundamental BEFORE vs AFTER Trigger & Alasan Penggunaannya

- Perbedaan Fundamental:
 - BEFORE Trigger: Dieksekusi sebelum operasi INSERT, UPDATE, atau DELETE benar-benar ditulis/disimpan ke dalam tabel database. Trigger ini dapat membatalkan operasi (*abort*) atau mengubah nilai data sebelum masuk ke tabel.
 - AFTER Trigger: Dieksekusi setelah operasi INSERT, UPDATE, atau DELETE berhasil dijalankan pada tabel. Trigger ini digunakan untuk aksi lanjutan (seperti *logging*, pembaruan tabel lain) karena data baris baru dipastikan sudah valid dan berhasil masuk.
- Mengapa Validasi Saldo Menggunakan BEFORE INSERT?
 - Untuk mencegah masuknya data transaksi yang tidak valid ke tabel transaksi. Jika saldo rekening pengirim tidak mencukupi, trigger BEFORE akan melempar *exception* yang langsung menggagalkan seluruh proses INSERT sebelum data tersebut sempat tersimpan di database.
- Mengapa Update Saldo Menggunakan AFTER INSERT?
 - Untuk menjaga konsistensi data. Pembaruan saldo di tabel rekening hanya boleh terjadi jika catatan transaksi di tabel transaksi sudah benar-benar valid dan dipastikan berhasil dimasukkan tanpa adanya error/kegagalan.

b. Flowchart Execution Flow (INSERT Transaksi → Update Saldo)

